



Skin Disease Classifier

Project Documentation

Name	Roll Number
Humna Imran	2023-BS-AI-017
Qandeel Asim	2023-BS-AI-037
Eman Tahir	2023-BS-AI-015
Ayesha Imran	2023-BS-AI-061

Deep Learning Ensemble for Automated Dermatological Image Classification

Project	Skin Disease Image Classifier
Framework	TensorFlow / Keras
Architecture	Multi-backbone CNN Ensemble
Environment	Google Colab / Local Jupyter
Status	Complete — Training + Inference Ready

1. PROJECT OVERVIEW

1.1 Purpose

This project implements a multi-backbone convolutional neural network (CNN) ensemble for automated skin disease classification from dermoscopic or clinical photographs. The system is designed to assist dermatologists and medical practitioners by providing ranked differential diagnoses from uploaded images, thereby supporting faster and more consistent clinical decision-making.

1.2 Key Objectives

- Train multiple CNN backbones on a labelled skin disease image dataset
- Apply class balancing, data augmentation, and two-stage fine-tuning for robust generalisation
- Combine individual model predictions via ensemble averaging to maximise accuracy
- Provide Test-Time Augmentation (TTA) to reduce prediction variance at inference
- Deliver an interactive Jupyter widget frontend for real-time clinical use

1.3 Scope

The notebook covers the full machine learning pipeline from raw image folders through to an interactive deployment-ready widget, including dataset validation, preprocessing, model training, evaluation, and inference. It is designed to run in Google Colab with GPU acceleration or on a local machine.

2. SYSTEM ARCHITECTURE

2.1 High-Level Pipeline

The system follows a standard supervised image-classification workflow with additional ensemble and TTA layers:

Pipeline Stage	Description
Stage 1	Dataset Validation & Directory Setup
Stage 2	Library Imports & GPU Configuration
Stage 3	Backbone Registry Definition
Stage 4	Data Augmentation & Dataset Construction
Stage 5	Class Weight Computation
Stage 6	Focal Loss Definition

Pipeline Stage	Description
Stage 7	Model Construction (build_model)
Stage 8	Two-Stage Training per Backbone
Stage 9	Evaluation: Per-model & Ensemble
Stage 10	TTA Inference Function
Stage 11	Model Reloading (inference-only session)
Stage 12	Interactive Widget Frontend

2.2 Ensemble Architecture

Each backbone independently produces a probability vector over all disease classes. At inference time the vectors are averaged across all loaded backbones and across all TTA views. A confidence threshold gates the final label: predictions below the threshold are reported as 'No Clear Disease'.

Inference Flow

```
Image Input
  ↓ load & resize to 224×224
  ↓ generate N TTA views (random augmentation)
  ↓ preprocess per backbone
  ↓ [ResNet50 | EfficientNetV2B0 | ...] predict each view
  ↓ average across TTA views (per backbone)
  ↓ average across backbones (ensemble)
  ↓ argsort -> Top-K labels + confidence scores
  ↓ apply threshold -> final output
```

3. DATASET & DIRECTORY STRUCTURE

3.1 Expected Folder Layout

The notebook expects images organised in a two-level class-folder hierarchy:

```
skin_app/
├── train/
│   ├── <class_A>/  <- one folder per disease
│   ├── <class_B>/
│   └── ...
├── test/
│   ├── <class_A>/
│   └── ...
└── checkpoints/  <- auto-created; stores .keras files
```

3.2 Validation Rules

- Both train/ and test/ must exist under DRIVE_ROOT
- The set of class sub-folders must be identical in both splits
- Mismatches raise a ValueError listing the discrepant class names

3.3 Dataset Split

Split	Source
Training Set	80% of images in train/ (Keras validation_split=0.2)
Validation Set	20% of images in train/ (same split, no shuffle)
Test Set	All images in test/ (no split; shuffle disabled)

3.4 Key Hyperparameters

Parameter	Value
IMG_SIZE	224 × 224 pixels
BATCH_SIZE	32
SEED	42 (for reproducible splits)
AUTOTUNE	tf.data.AUTOTUNE (adaptive parallelism)

4. DATA AUGMENTATION

4.1 Augmentation Pipeline

A sequential Keras augmentation layer is applied only during training. The pipeline introduces controlled geometric and photometric variation to improve generalisation and reduce overfitting:

Augmentation Layer	Effect
RandomFlip	Horizontal flips with 50% probability
RandomRotation	±10% of 2π rotation
RandomZoom	±15% zoom in/out
RandomTranslation	±10% shift in both x and y axes
RandomContrast	±15% contrast adjustment

4.2 Test-Time Augmentation (TTA)

During inference the same augmentation pipeline is applied N times (default `n_aug = 5`) to generate multiple stochastic views of each input image. Predictions across views are averaged before ensemble aggregation. TTA reduces prediction variance and typically improves accuracy by 1-3 percentage points on uncertain samples.

5. MODEL ARCHITECTURES

5.1 Backbone Registry

Four pre-trained ImageNet backbones are registered. Two are active by default (ResNet50 and EfficientNetV2B0). Any combination can be selected via the `ACTIVE_BACKBONES` list:

Backbone Key	Architecture	Unfreeze Last N	Status
resnet50	ResNet-50	50 layers	Active (default)
efficientnetv2b0	EfficientNetV2-B0	40 layers	Active (default)
densenet121	DenseNet-121	40 layers	Inactive (optional)
mobilenetv3large	MobileNetV3-Large	30 layers	Inactive (optional)

5.2 Custom Classification Head

Each backbone is augmented with the same lightweight classification head:

```
[Backbone CNN] -> GlobalAveragePooling2D
                 -> BatchNormalization
                 -> Dense(256, relu, L2=1e-4)
                 -> Dropout(0.5)
                 -> Dense(NUM_CLASSES, softmax, dtype=float32)
```

The final Dense layer is explicitly cast to float32 to ensure numerical stability when mixed-precision (float16) training is active on GPU.

6. TRAINING PROCEDURE

6.1 Two-Stage Fine-Tuning

Training follows a two-stage transfer-learning strategy that is standard practice for medical imaging tasks:

Stage	Description
Stage 1 — Head Training	Backbone frozen entirely; only the custom classification head is trained. Learning rate: 1e-3 (cosine decay). Epochs: 20. Allows the head to converge to a reasonable feature space before backbone weights are adjusted.
Stage 2 — Fine-Tuning	Top N layers of the backbone are unfrozen; full network trained at a much lower learning rate: 1e-5 (cosine decay). Epochs: 12. Adapts high-level backbone features to the skin-disease domain while preserving low-level ImageNet representations.

6.2 Optimizer & Scheduler

Parameter	Value
Optimizer	AdamW (Adam with decoupled weight decay)
Stage 1 LR	1e-3, CosineDecay over stage-1 steps
Stage 2 LR	1e-5, CosineDecay over stage-2 steps
Weight Decay (S1)	1e-4
Weight Decay (S2)	1e-5

6.3 Loss Function

Stage 1 and Stage 2 both use Categorical Cross-Entropy with label smoothing (0.05). Label smoothing prevents over-confident probability estimates and acts as a regulariser. An alternative Focal Loss implementation (gamma=2.0) is also defined in the notebook for experimentation with hard-sample mining.

6.4 Class Imbalance Handling

Balanced class weights are computed via `sklearn.utils.class_weight.compute_class_weight` with `strategy='balanced'`. Weights are passed to `model.fit()` as the `class_weight` argument, causing the optimizer to penalise errors on minority classes proportionally more than errors on majority classes.

6.5 Callbacks

Callback	Behaviour
EarlyStopping	Monitors <code>val_loss</code> ; stops training if no improvement for 6 epochs; restores best weights on stop
ModelCheckpoint	Saves the best model (by <code>val_loss</code>) to <code>checkpoints/best_<backbone>.keras</code> after every epoch

6.6 Mixed Precision

When a GPU is detected, the global Keras policy is set to 'mixed_float16', storing weights in float16 for reduced memory and faster matrix multiplications, while accumulating gradients in float32. A bug-fix cast (tf.cast to float32 before preprocessing) prevents half-precision values from leaking into the preprocessing arithmetic.

7. EVALUATION METHODOLOGY

7.1 Metrics

- Test Accuracy: fraction of correctly predicted test images per model and for the ensemble
- Classification Report: per-class precision, recall, F1-score, and support (via sklearn)
- Confusion Matrix: heatmap of actual vs. predicted classes on the test set

7.2 Evaluation Flow

After training all active backbones, evaluation proceeds as follows:

- Predict probability vectors for the full test set with each trained model
- Average vectors across models to produce ensemble probabilities
- Compute and display accuracy for each individual backbone and the ensemble
- Print the full sklearn classification_report for the ensemble predictions
- Plot a colour-coded confusion matrix for visual error analysis

7.3 Training Visualisation

For each backbone and each training stage, accuracy and loss curves are plotted side-by-side using Matplotlib. These plots allow the practitioner to diagnose overfitting, under-fitting, or training instability at a glance.

8. INFERENCE & PREDICTION API

8.1 predict_topk()

The primary inference function for the deployment cell. Returns the top-K ranked predictions with confidence scores.

Parameter	Description
image_path	str — path to any image file readable by TensorFlow
top_k	int (default 5) — number of ranked predictions to return
n_aug	int (default 5) — number of TTA augmentation views

Parameter	Description
threshold	float (default 30.0) — minimum confidence % for a valid diagnosis

Returns: list of (label: str, confidence: float) tuples sorted by confidence descending. If the top-1 confidence is below threshold, the label is replaced with 'No Clear Disease'.

8.2 predict_skin_disease_tta() (training notebook)

A simplified inference function defined in the training notebook. Returns only the single top-1 prediction and confidence, with the same TTA and threshold logic as predict_topk(). Suitable for quick command-line testing.

8.3 Inference-Only Session

The notebook includes a dedicated Model Reloading cell that reconstructs the trained dictionary from saved .keras checkpoints without re-running the training pipeline. This allows the inference widget to be launched in a fresh kernel or shared runtime by executing only two cells (reload + widget).

- Priority: final_<backbone>.keras (full post-fine-tuning model)
- Fallback: best_<backbone>.keras (best validation checkpoint)

9. INTERACTIVE WIDGET FRONTEND

9.1 Overview

The final notebook cell renders a polished ipywidgets-based user interface directly inside the Jupyter/Colab output pane. It allows non-technical users to upload dermoscopic images and receive ranked predictions without writing any code.

9.2 Widget Components

Component	Function
Title Banner	Displays project name, number of loaded backbones, and feature flags (TTA, Top-K)
Top-K Slider	Adjusts the number of disease candidates shown (1 to min(10, NUM_CLASSES))
TTA Views Slider	Controls the number of augmentation views for TTA inference (1–15)
Confidence Threshold	Sets the minimum confidence (%) for a valid label; below this shows 'No Clear Disease'
File Upload Button	Accepts one or more image files (accepts image/*)

Component	Function
Run Predictions Button	Triggers inference on all uploaded images
Output Pane	Renders side-by-side: uploaded image + horizontal confidence bar chart
Status Label	Real-time status messages (uploading, processing, done, errors)

9.3 Results Visualisation

For each uploaded image a Matplotlib figure is rendered with two panels. The left panel shows the original image with its filename as a caption. The right panel shows a horizontal bar chart of the top-K predictions in descending confidence order. Bars are colour-coded using a blue palette; the top-1 prediction is highlighted in bold with a star prefix. Confidence percentages are annotated on each bar.

9.4 Colour Palette

Element	Colour
Primary / Accent	#2A6EBB (deep blue)
Bar Palette	#2A6EBB, #4A9FD4, #7DC3E8, #A8D8EA, #C9EBF5
Background	#F7FAFB
Card Background	#FFFFFF
Body Text	#1A2332

10. DEPENDENCIES & ENVIRONMENT

10.1 Python Libraries

Library	Purpose
tensorflow	Core deep learning framework; model building, training, inference
tensorflow.keras	High-level API for layers, datasets, callbacks, preprocessing
numpy	Numerical array operations
pandas	Class distribution analysis and result tabulation
matplotlib	Training history plots and result visualisations
seaborn	Confusion matrix heatmap and bar plots
scikit-learn	Class weight computation, classification report, confusion matrix
ipywidgets	Interactive Jupyter widget frontend

Library	Purpose
IPython	Rich display helpers for widget rendering
os, io, base64	Standard library: file I/O, image encoding

10.2 Hardware Requirements

Environment	Note
Recommended	GPU (NVIDIA, CUDA-capable) — required for mixed_float16 and acceptable training time
Minimum	CPU only (training will be very slow; inference usable)
Google Colab	Runtime > Change runtime type > T4 GPU (free tier sufficient)
VRAM	4 GB+ recommended for batch_size=32 with dual backbones

10.3 Compatibility Notes

- The notebook auto-detects Colab vs. local execution; DRIVE_ROOT adjusts accordingly
- Mixed precision is only activated when at least one GPU is detected
- A float32 cast bug-fix ensures preprocessing never receives float16 tensors
- ImageNet weight downloads include a retry loop (max 3 attempts) for flaky Colab connections

11. CONFIGURATION REFERENCE

11.1 Global Constants

Constant	Description
DRIVE_ROOT	Root path to skin_app/ folder (auto-set for Colab; override for local)
TRAIN_DIR	DRIVE_ROOT/train
TEST_DIR	DRIVE_ROOT/test
CHECKPOINT_DIR	DRIVE_ROOT/checkpoints (auto-created)
IMG_SIZE	224 (input image resolution in pixels)
BATCH_SIZE	32
SEED	42
ACTIVE_BACKBONES	["resnet50", "efficientnetv2b0"] by default
EPOCHS_STAGE1	20
EPOCHS_STAGE2	12

11.2 How to Add a New Backbone

To register an additional pre-trained backbone, add an entry to `BACKBONE_REGISTRY` with three keys:

- `builder`: the Keras Applications class constructor
- `preprocess`: the corresponding `preprocess_input` function
- `unfreeze_last_n`: number of layers from the end to unfreeze in Stage 2

Then add the backbone key string to `ACTIVE_BACKBONES` and re-run the training cells.

12. KNOWN ISSUES & LIMITATIONS

12.1 Technical Notes

Issue	Detail
Mixed-precision bug-fix	Without the explicit <code>tf.cast(x, tf.float32)</code> before <code>preprocess_input</code> , certain preprocessing functions receive float16 inputs and produce NaN gradients. This has been fixed in the code.
ImageNet download retries	Colab's network can intermittently fail during keras weight downloads. The <code>load_pretrained_backbone()</code> wrapper retries up to 3 times before raising a descriptive error.
Colab session reset	Models must be reloaded from checkpoints after any Colab session restart. Use the dedicated Model Reloading cell.
Widget multi-upload	The FileUpload widget delivers files as a dict; the code handles both dict (newer ipywidgets) and bytes (older) meta formats for broad compatibility.

12.2 Limitations

- The system is not a certified medical device and should be used only as a decision-support tool
- Classification performance is bounded by dataset quality and class coverage
- Very rare or visually ambiguous conditions may yield low confidence scores
- The confidence threshold (default 30%) requires calibration on a representative validation set for clinical deployment
- No DICOM support; input must be standard image formats (JPEG, PNG, BMP)

13. FUTURE IMPROVEMENTS

13.1 Short-Term

- Hyperparameter search (learning rate, batch size, dropout) with Keras Tuner or Optuna
- Integrate Grad-CAM / SHAP saliency maps for explainability of predictions
- Add ROC-AUC and per-class AUROC to the evaluation metrics
- Export models to TensorFlow Lite or ONNX for mobile deployment

13.2 Medium-Term

- Integrate all four registered backbones into the active ensemble for higher accuracy
- Calibrate prediction probabilities using Platt scaling or temperature scaling
- Implement uncertainty quantification via Monte Carlo Dropout
- Add a Streamlit or Gradio web interface for browser-based clinical access

13.3 Long-Term

- Fine-tune on a multi-site clinical dataset with patient demographic metadata
- Develop a REST API with FastAPI or Flask for integration with hospital information systems
- Conduct prospective clinical validation studies with dermatologist oversight
- Pursue regulatory certification (FDA 510(k) or CE marking) for clinical use

14. GLOSSARY

Term	Definition
Backbone	A pre-trained CNN used as a feature extractor (e.g. ResNet50)
Transfer Learning	Reusing weights trained on a large dataset (ImageNet) as a starting point for a different task
Fine-Tuning	Unfreezing and retraining some or all backbone layers at a low learning rate
TTA (Test-Time Aug.)	Applying random augmentations at inference time and averaging predictions to reduce variance
Ensemble	Combining predictions from multiple models to produce a single, more accurate output
Focal Loss	A loss function that down-weights easy examples, focusing training on hard misclassified samples
Label Smoothing	Replacing one-hot targets with soft targets (e.g. 0.95/0.05) to reduce over-confidence
Class Weight	Per-class scalar multiplied with the loss to compensate for imbalanced datasets
AdamW	Adam optimiser with decoupled weight decay; prevents weight decay from interacting with adaptive gradients
CosineDecay	Learning rate schedule that anneals the LR following a cosine curve from initial to zero

Term	Definition
Mixed Precision	Training with float16 arithmetic and float32 gradient accumulation for speed and memory efficiency
Confidence Threshold	Minimum predicted probability (%) required to report a disease label rather than 'No Clear Disease'

End of Document